

Scenario 1:

Scenario: You are a junior developer at a fintech startup building a digital wallet app. The team is debating between Waterfall and Agile methodology. The project has frequently changing requirements and the client expects to review working features every two weeks.

Question 1: Which development methodology would you recommend for this project, and why? Identify two specific risks the team would face if they chose the wrong methodology for this context.

Answer:

I would recommend Agile because the requirements are changing regularly and the client wants to see working features every two weeks. With Agile, the team can work in short sprints, show the client what has been completed, take feedback, and make changes in the next sprint. This is better than Waterfall because Waterfall works best when requirements are already clear and do not change much.

If the team chooses Waterfall, one risk is that a requirement may change after a lot of design or coding is already finished. This can cause rework, extra cost, and delays. Another risk is that the client may only see the final product very late, so the team may realise too late that the features are not what the client expected.

Scenario 2:

Scenario: You are collaborating on a college project website with three teammates. Just before a client demo, one teammate pushed incomplete code directly to the main branch, breaking the layout for everyone else on the team.

Question : Describe the Git workflow your team should adopt to prevent this from happening again. Name at least two Git commands your team should use as part of this workflow and explain the role of each command.

Answer:

Our team should use a feature branch workflow. Nobody should directly push unfinished work to the main branch. Each person should create a separate branch for their task, such as a navbar branch or contact-form branch. When the work is ready, they should create a pull request. Another teammate can review it and test the website before it is merged into main. The main branch should also be protected so direct pushes are not allowed.

`git switch -c feature-name` is used to create and move to a new branch. This keeps unfinished work separate from the main branch.

`git add .` is used to stage changed files before committing them.

`git commit -m "message"` saves a checkpoint with a short description of the changes.

`git push -u origin feature-name` uploads the branch to GitHub so that a pull request can be created. The pull request should be reviewed before merging into main.

Scenario 3:

Scenario: You are building an online internship application form for a college placement portal. The form must collect: applicant name, email address, phone number, preferred domain (one of: Web, Data, Mobile, AI), and a resume file upload. The institution requires that no field be left empty and that the email field only accepts valid email formats — without writing any JavaScript.

Question : List the appropriate HTML input types you would use for each of the five fields. Explain how you would use HTML5 built-in validation attributes to enforce the two constraints mentioned, and why these attributes alone are not sufficient for production-level validation.

Answer:

The applicant name can use `<input type="text">`. The email field should use `<input type="email">`. The phone number can use `<input type="tel">`. For the preferred domain, I would use a `<select>` element with the options Web, Data, Mobile, and AI because the user only has to choose one option. For the resume, I would use `<input type="file">`.

I would add the `required` attribute to every field. This stops the form from being submitted when any field is empty. For the email field, `type="email"` checks whether the value looks like an email address. For the domain select box, the first option can be something like “Select a domain” with an empty value, so it will not pass validation unless a real option is selected.

These checks are helpful, but they are only browser-side checks. A user can bypass them or send data directly to the server. So, in a real application, the server must validate the data again. It should also check the resume file type and size before saving it.

Scenario 4:

Scenario: You are designing a news portal that displays articles in a 3-column grid on desktop screens and in a single-column layout on mobile devices. Your team wants to use Bootstrap's grid system to achieve this without writing any custom media query CSS.

Question : Explain how Bootstrap's grid class system works to produce this responsive layout. Write the Bootstrap class combination you would apply to the article column div, and explain what each class in the combination controls.

Answer:

Bootstrap uses a 12-column grid system. We first put the article cards inside a row, then give each article column responsive classes. I would use:

```
<div class="col-12 col-md-4">
```

col-12 means the article takes all 12 columns by default. This is useful on mobile because each article gets its own row, giving a single-column layout.

col-md-4 starts working from the medium screen size and above. It gives each article 4 out of 12 columns. Since $4 + 4 + 4 = 12$, three articles fit in one row on desktop screens. Bootstrap already includes the needed responsive CSS, so no custom media query is needed.

Scenario 5:

Scenario: You are writing a C program to record daily rainfall data for a month (30 days). You need to store all 30 readings, calculate the monthly average, and then identify which specific days recorded rainfall above that average.

Question : Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the above-average days — and explain why both steps cannot be done in a single loop.

Answer:

An array is the correct choice because we have 30 values of the same type. For example, we can use `float rainfall[30];`. Each position in the array represents one day. This is much easier than making 30 different variables because loops can be used to enter, calculate, and compare all the values. It also makes the program shorter and easier to update.

First, the program should use a loop to add all 30 rainfall values into a total. Then it should calculate the average by dividing the total by 30.

After that, the program should use another loop. In this loop, it checks every value. If `rainfall[i] > average`, then it prints the day number and the rainfall value.

We need two loops because the final average is not known until all 30 values have been added. If we check a day before the average is calculated, we cannot correctly decide whether it is above average or not.

Scenario 6:

Scenario: You are debugging a C program that uses a pointer to traverse a string entered by the user. The program works correctly when the user types a name, but crashes immediately when the input field is left empty (the user just presses Enter).

Question : Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch?

Answer:

When the user presses Enter without typing anything, the string is empty. In C, an empty string starts with the null character `\0`. The crash usually happens because the program tries to read a character even though it has already reached the null character, or because the pointer is not pointing to valid memory.

Before dereferencing a pointer, I would check that the pointer is not NULL and that the current character is not the null character. For example: `while (p != NULL && *p != '\0') { p++; }`

In pointer traversal, we move through the string using the pointer itself, for example `p++`, and read characters using `*p`. In array index traversal, we keep the array name and use an index, for example `name[i]`, then increase `i`.

Both methods work, but array indexing is usually easier for beginners because it is easy to see the first position, like `name[0]`. This can make the empty-string check easier to understand. Pointer traversal is useful too, but the checks need to be written carefully.